

Enterprise Salesforce Defect Leakage Predictor Agent

GenAI + ML Powered QA Risk Intelligence Platform

Prepared for Mentor Review & Enterprise QA Architecture Discussion This document outlines the proposed design, architecture, workflow, and implementation approach for an enterprise-grade Salesforce Defect Leakage Predictor Agent.

1. Executive Summary

This proposal presents an enterprise-grade AI-powered Defect Leakage Predictor Agent designed for Salesforce ecosystems including Sales Cloud (CPQ & QTC) and Service Cloud. The platform aims to proactively identify high-risk Jira user stories, predict potential production defect leakage, and generate explainable QA validation intelligence before deployment. The solution combines: • Retrieval-Augmented Generation (RAG) • Machine Learning-based Risk Scoring • Historical Defect Intelligence • Salesforce Apex Change Analysis • LLM-driven QA Recommendations

2. Business Problem

Enterprise QA teams often face: • Production defect leakage • Limited regression coverage • Complex Apex dependency impacts • Insufficient visibility into deployment risks • Repeated historical defect patterns Current testing approaches are reactive instead of predictive. The proposed platform transforms QA into predictive quality engineering.

3. Project Vision

Build an intelligent QA risk intelligence platform that: • Predicts defect leakage risk before deployment • Analyzes Apex code changes • Retrieves similar historical production defects • Recommends targeted regression validation • Generates leadership-ready risk reports

4. Enterprise Systems & Data Sources

The proposed solution integrates with: • Jira - User Stories - Production Defects - Root Causes - Severity & Priority • GitLab - Pull Requests - Apex Class Changes - Deployment Metadata • Salesforce CLI - Apex Metadata - Changed Components - Deployment Intelligence • Historical Defect Dashboards - Escaped Defects - Regression Failures - Similar Defect Patterns

5. Available Enterprise Data

Available datasets include: • Test case IDs • Pass/fail history • Apex codebase access • Severity and priority data • Root cause information • Sprint-level story mapping • Historical defect-user story linkage • Deployment change details

6. Defect Predictor Workflow

Step 1: Read Jira User Story and associated deployment details. Step 2: Extract impacted Apex classes and Salesforce components. Step 3: Retrieve similar historical production defects using RAG. Step 4: Calculate defect

leakage risk score. Step 5: Generate explainable QA validation report. Step 6: Provide targeted regression recommendations to QA teams.

7. Proposed AI Architecture

Architecture Layers: 1. Data Ingestion Layer Jira + GitLab + Salesforce CLI 2. Retrieval Layer (RAG) Historical defect similarity search 3. Rules Engine Salesforce-specific risk heuristics 4. ML Risk Engine Leakage probability scoring 5. LLM Intelligence Layer Root-cause explanation QA recommendations Leadership summaries

8. Why RAG + ML + Rules Instead of Pure LLM

The enterprise environment currently has: • Strong historical defect linkage • Moderate traceability • Limited automated test mapping Because of this, the recommended architecture is: Rules + RAG + Lightweight ML + LLM Explanation Benefits: • More explainable predictions • Faster implementation • Lower hallucination risk • Better enterprise governance

9. Key Prediction Signals

Important prediction features include: • Apex classes modified • Historical defects linked to component • Severity history • Deployment scope • Root-cause patterns • Similar historical user stories • Sprint-level change complexity

10. Example Output Report

Risk Score: 82% Likely Leakage Areas: • Discount Calculation • Quote Amendment Flow • Contract Renewal Sync Historical Similar Defects: • DEF-4432 • DEF-5121 Recommendation: Perform additional regression testing before deployment. Potential Severity: High

11. Enterprise Use Cases

The platform can support: • Deployment risk analysis • Predictive regression testing • QA prioritization • Leadership risk dashboards • Historical defect intelligence • Faster release confidence

12. Expected Business KPIs

Expected measurable outcomes: • 20% reduction in production defect leakage • 30% test optimization • Improved release confidence • Faster QA decision-making • Reduced regression effort

13. Phase 1 MVP Scope

Recommended MVP capabilities: • Jira integration • Apex change parser • Historical defect retrieval • Risk scoring engine • LLM-based explanation generator • PDF/HTML QA risk reports

14. Future Roadmap

Phase 2: • Advanced ML models • Test impact analysis • Azure deployment • Slack/Teams notifications Phase 3: • Autonomous regression optimization • AI release governance • Intelligent release summaries

15. Conclusion

The proposed Defect Leakage Predictor Agent is an enterprise-ready GenAI for QA initiative focused on predictive quality engineering. Instead of reacting to production failures, the platform enables organizations to proactively identify risk before deployment and optimize testing efforts using AI-driven intelligence.

16. High-Level Architecture Overview

Layer	Purpose
Jira Integration	Fetch user stories, defects, root causes
GitLab Analysis	Identify Apex code changes
Salesforce CLI	Extract impacted metadata
RAG Retrieval Engine	Find historical similar defects
ML Risk Engine	Generate leakage probability
LLM Layer	Generate QA recommendations
Reporting Layer	Leadership & QA reports

This proposal can serve as the foundation for: • Enterprise PoC • Internal Innovation Initiative • Hackathon Submission • QA Transformation Program • GenAI for QA Accelerator